La Plateforme
La grande école du numérique pour tous

java Abstraction jour 3

Sommaire

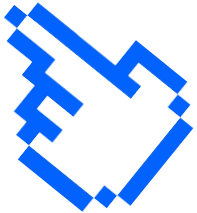
3 Polymorphisme

3 Abstraction

5 Interfaces

7 Méthodes virtuelles

12 Questions



La Plateforme
La grande école du numérique pour tous

le polymorphisme

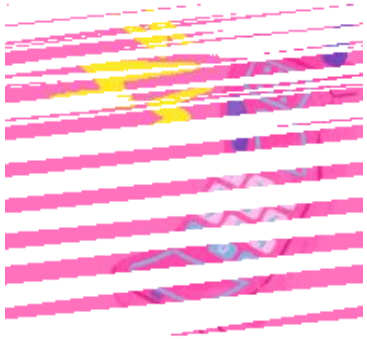
Le polymorphisme

= C'est la **capacité** d'un objet à prendre **plusieurs formes**

permet de "**redéfinir**" les **méthodes** de la classe **mère** donc redéfinir leurs **comportements sans changer leur signature**.

= superclasse commune

exemple concret

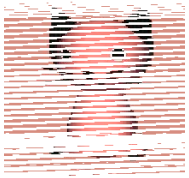


Vous disposez d’un smartphone pour communiquer.

Le mode de communication que vous choisissez peut être n’importe quoi. Il peut s'agir d'un appel, d'un SMS, d'un message photo, mail, etc.

Ainsi, l'objectif est commun, c'est-à-dire la communication, mais leur approche est différente. C'est appelé **Polymorphisme.**

Règles de remplacement de méthode



- La **signature** de la méthode (son nom), la **liste** des **paramètres** et le **type** de **retour**, doivent **correspondre exactement**.

- La **méthode surchargée** peut **élargir l'accessibilité** mais **pas** la **restreindre**, c'est-à-dire que si elle est privée dans la classe de base, la classe enfant peut la rendre publique mais pas l'inverse.

Avantages

Flexibilité et extensibilité :

- ➔ Traitement des objets de différentes classes de manière uniforme via une **interface commune.**
- ➔ **Ajout facile** de nouvelles fonctionnalités en utilisant des interfaces ou des classes abstraites
- ➔ **Réutilisation** des méthodes

Simplification du Code :

- ➔ **Réduction du code conditionnel** : comme if-else ou switch-case pour différencier les actions en fonction du type d'objet.
- ➔ **Clarté et lisibilité**

Maintenance et Évolution du Code :

- ➔ **Facilité de maintenance**
- ➔ **Isolation des changements** : Cela rend le code plus robuste et moins susceptible de contenir des bogues introduits par des modifications.

Polymorphisme vs Héritage

	Héritage	Polymorphisme
Définition	Une classe enfant hérite des attributs et méthodes d'une classe mère	Un objet peut prendre plusieurs formes et se comporter différemment selon son type réel
Mot clé	extends	@Override
But	Réutiliser du code existant	Adapter un comportement selon le contexte
Quand ?	À la conception des classes	À l'exécution (liaison dynamique)
Question posée	"Est-ce que B *est un* A ?"	"Comment B se comporte-t-il *en tant que* A ?"
Exemple	Chien extends Animal	chien.deplacer() → "Je marche" au lieu de "Je me déplace"
Sens	Vertical (hiérarchie)	Horizontal (comportements différents au même niveau)

Le polymorphisme

```java
class Chien extends Animal {
    void deplacer() {
        System.out.println("Je marche");
    }
}
```

```java
class Animal {
    void deplacer() {
    System.out.println("Je me déplace");
}
```

```java
class Oiseau extends Animal {
    void deplacer(){
        System.out.println("Je vole");
    }
}
```

Le mot clé **@Override** (java 5) n'est pas obligatoire mais recommandé si vous utilisez @Override sur une méthode qui n'est pas redéfinie, une erreur sera signalée à la compilation.

Le polymorphisme : résultat

```java
public class Test {
public static void main(String[] args) {
    Animal a1 = new Animal();
    Animal a2 = new Chien();

    a1.deplacer();
    a2.deplacer();
    }
}
```

```
Je me déplace
Je marche
```

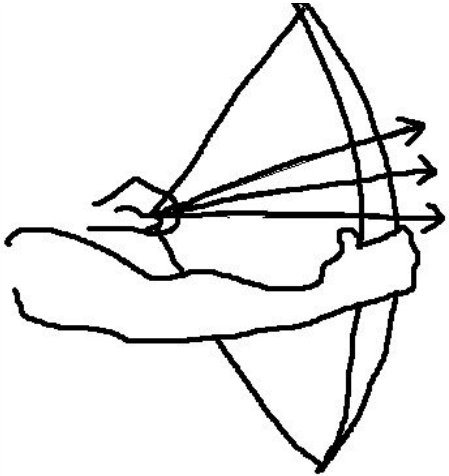
Garder le comportement de la méthode mère

Pour, non pas totalement réécrire la méthode, mais bénéficier du comportement de la méthode de base **et** y adjoindre quelques instructions supplémentaires : il suffit simplement d'appeler la méthode de base **dans** la méthode redéfinie.

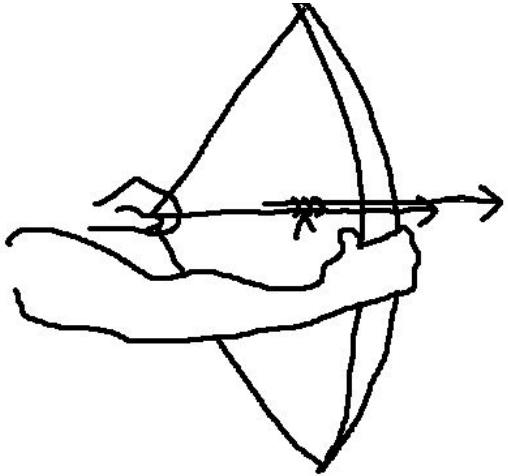
Comment ? À l'aide du mot-clé **super**. super fait référence à la classe mère et permet d'appeler ses méthode

```java
public class Voiture extends Vehicule {

    public Voiture(String marque, String modele) {
        super(marque, modele);
    }

    @Override
    public void seDeplacer() {
        super.seDeplacer();
        System.out.println("Et c'est une voiture !");
    }
}
```

surcharge vs redéfinition



Overloading



Overriding

Méthode surchargée : plusieurs fois la même méthode dans une même classe, avec le même nom mais avec des paramètres différents.

Méthode redéfinie : une méthode dans une classe fille qui porte le même nom et possède les mêmes paramètres qu'une méthode de sa classe mère.

La surcharge

```java
public class Calculatrice {
    public int diviser(int num, int denom) {
        return num / denom;
    }

    public double diviser(double num, double denom) {
        return num / denom;
   }
}
```

À la **compilation**, le programme déterminera quelle méthode appeler, en fonction du type / nombre / ordre des arguments transmis lors de l'appel.

La Plateforme
La grande école du numérique pour tous

Abstraction

/ Classes abstraites

- ne peut **pas** être **instanciée**
- si elle **possède** au moins **une méthode abstraite**
- des **schémas** pour des classes enfant

```java
abstract class Animal {
    public abstract void makeSound();

    public void sleep() {
        System.out.println("L'animal dort.");
    }
}

class Dog extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Le chien aboie.");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.makeSound();
        dog.sleep();
    }
}
```

classe abstraite

méthode abstraite

méthode concrète

Permet d'indiquer au compilateur que cela est bien une Override

- peut avoir des constructeurs
- peut avoir des attributs d'instance
- méthodes acceptées : abstraites et concrètes

La Plateforme
La grande école du numérique pour tous

Interface

Interface

Une interface est une classe abstraite dont TOUTES les méthodes sont abstraites.

C'est donc un contrat pour réaliser différentes classes qui peuvent être utilisées de la même manière, et interchangeables.

```java
interface Animal {
  public void animalSound();
  public void sleep();
}

class Pig implements Animal {
  public void animalSound() {
    System.out.println("The pig says: wee wee");
  }
  public void sleep() {
    System.out.println("Zzz");
  }
}

class Main {
  public static void main(String[] args) {
    Pig myPig = new Pig();
    myPig.animalSound();
    myPig.sleep();
  }
}
```

- ➔ pas de constructeurs
- ➔ que des constantes

Toutes les variables dans une interface sont automatiquement :

- public
- static
- final

La déclaration de variable n'est pas conseillée dans une interface !

Le default dans les interfaces

- **default** : permet déclarer une méthode complète, l'override n'est pas obligatoire mais possible
  - **Pas d'état** (pas d'attributs d'instance)
  - **Pas de constructeur**
  - **Héritage multiple** toujours possible

```java
public interface Animal {
    void makeSound();

    default void breathe() {
        System.out.println("Je respire ... ");
    }
}
```

le principe de la compatibilité ascendante : **le code existant doit continuer à fonctionner après une mise à jour**, sans modification.

Le cas d'usage principal était la **compatibilité ascendante** : quand Java a voulu ajouter des méthodes aux interfaces existantes (comme List, Collection), il était impossible de le faire sans casser tout le code existant. Les méthodes default ont résolu ce problème.

Par exemple, List.sort() est une méthode default ajoutée en Java 8.

Java tourne dans des contextes critiques (banques, hôpitaux, grandes entreprises) où **mettre à jour du code legacy est extrêmement coûteux**.

La Plateforme
La grande école du numérique pour tous

Méthodes virtuelles

Méthodes virtuelles

- ➔ **Toutes** les méthodes **non static, final, ou private** sont virtuelles par défaut.
- ➔ Ces méthodes peuvent être **redéfinies / surchargées** dans les sous-classes.

Lorsque vous appelez une méthode sur une instance d'une classe dérivée, Java utilise la **liaison dynamique** pour déterminer quelle implémentation de la méthode doit être exécutée, en fonction du type réel de l'objet.

Liaison statique (à la compilation)

Pour les méthodes static, final, private, Java sait déjà à la compilation quelle méthode appeler. C'est rapide, prévisible, mais pas flexible.

Liaison dynamique (à l'exécution)

Pour toutes les autres méthodes (virtuelles), Java attend l'exécution pour décider.

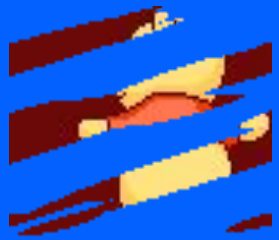
final = constante

Classe abstraite vs Interface

```java
abstract class Animal {
    abstract void makeSound();

    void sleep() {
        System.out.println("Sleeping ... ");
    }
}

class Dog extends Animal {
    void makeSound() {
        System.out.println("Bark");
    }
}

public class AbstractClassExample {
    public static void main(String[] args) {
        Animal myDog = new Dog();
        myDog.makeSound();
        myDog.sleep();
    }
}
```

```java
interface Vehicle {
    void start();
    void stop();
}

class Car implements Vehicle {
    public void start() {
        System.out.println("Car starting");
    }

    public void stop() {
        System.out.println("Car stopping");
    }
}

public class InterfaceExample {
    public static void main(String[] args) {
        Vehicle myCar = new Car();
        myCar.start();
        myCar.stop();
    }
}
```

Des Questions ?



La Plateforme